```
        {
            cout<<"Parametrerized constructor";
             i=a;
              j=b'
         }
        Data(Data &a)
         {
            cout<<"Copy constructor";
             i=a.i;
              j=b.j'
         }

          void display()            //member function
           {
             cout<< i<<j;
           }
      };
int main()
{
          Data  d1,d2(10,20),d3(d2);
          d1.display();
          d2.display();
          d3.display();
           return 0;

}
```

தானமைவு ஆக்கியின் தனிச்சிறப்பு :

தானமைவு ஆக்கிகள், குறிப்பிட்ட தொடக்க மதிப்பு இல்லாத பொருள்களை உருவாக்கப் பயன்படுகிறது. பொருள்களின் அணியை உருவாக்கவும் இது பயன்படுகிறது.

14.8 ஆக்கிகளை அழைத்தல்

இரண்டு வழிகளில் அளபுருக்களை ஏற்கும் ஆக்கியைப் பயன்படுத்திப் பொருளை உருவாக்கலாம்.

1. உள்ளார்ந்த அழைப்பு (Implicit call)
2. வெளிப்படையான அழைப்பு (Explicit call)

14.8.1 உள்ளார்ந்த அழைப்பு:- (Implicit call)

இந்த முறையில், எப்பொழுதெல்லாம் பொருள் உருவாக்கப்படுகிறதோ அப்பொழுது அளபுருக்களை ஏற்கும் ஆக்கி தானாகவே அழைக்கப்படுகிறது. எடுத்துக்காட்டாக simple s1 (10, 20); இதில் s1 என்ற பொருளை உருவாக்கும்போது அளபுருவை ஏற்கும் ஆக்கி தானாகவே அழைக்கப்படுகிறது.

14.8.2 வெளிப்படையான அழைப்பு :- (Explicit call)

இந்த முறையில் ஆக்கியின் பெயரை வெளிப்படையாக கொடுக்கும்போது அளபுருக்களை ஏற்கும் ஆக்கி அழைக்கப்படுகிறது. அதனால் பொருளை உருவாக்கி தொடங்கி வைக்க முடிகிறது.

எடுத்துக்காட்டாக,

```
simple s1=simple(10,20);        //வெளிப்படையான அழைப்பு
```

வெளிப்படையான அழைப்பு முறை மிகவும் பொருத்தமானது. ஏனென்றால் இதில் தற்காலிக பொருளை உருவாக்குவதால் தரவு இழப்புக்கான வாய்ப்பு ஏற்படாது. தற்காலிக பொருள் கோவையில் பயன்படுத்தும் வரை நினைவகத்தில் இருக்கும். அதன்பிறகு அது அழிந்து விடும்.

Chapter 14.indd 242 4/19/2020 4:16:06 PM